Geldium Delinquency Prediction - Predictive Model Plan

Use this template to structure your submission. You can copy and paste content from GenAI tools and build around it with your own analysis.

1. Model Logic (Generated with GenAI)

Use a GenAI tool (e.g., ChatGPT, Gemini) to generate the logic or structure of your predictive model.
- You may include pseudo-code, a step-by-step process, or a simplified code snippet.
- Briefly explain what the model is designed to do.

Paste your GenAI-generated output below or describe the logic in your own words:

Objective: build a supervised binary-classification model that estimates the probability that a Geldium customer account becomes delinquent (Delinquent_Account = 1) at the next review, so that collections and retention outreach can be prioritised by risk. The logic below was drafted with a GenAI assistant and reviewed/refined by the analyst against the Task 1 EDA evidence (500 customers, 16.0% delinquent).

Step-by-step logic:

1. Data contract: use only pre-outcome features (demographics, credit behaviour, account attributes, 6-month payment history). Exclude the target and any field recorded after the delinquency outcome to prevent leakage.

2. Ingest and clean: apply the Task 1 cleaning log - standardise Employment_Status labels (245 rows), create quality flags (Credit_Utilization > 1; tenure-age inconsistency), derive payment-history aggregates (N_Missed_6M, N_Late_6M, Payment_Severity_Score).

3. Split once: stratified 80/20 hold-out test set; all tuning and preprocessing inside 5x3 repeated stratified cross-validation on the training fold only.

4. Preprocess inside each fold: median imputation + missingness indicators (Income 7.8%, Loan_Balance 5.8%, Credit_Score 0.4%) and one-hot encoding of categoricals - refitting imputation per fold to avoid leakage.

5. Model candidates: (i) dummy baseline for the no-skill floor, (ii) class-weighted logistic regression as the transparent champion, (iii) histogram gradient boosting as the non-linear challenger.

6. Tune by cross-validated PR-AUC (primary), subject to acceptable calibration; calibrate probabilities (Platt/isotonic) if they feed expected-loss estimates.

7. Set the operating threshold from a business cost matrix (cost of a missed delinquent = expected loss; cost of a false alert = outreach effort).

8. Gate and audit: fairness audit across Age bands, Location and Employment_Status; go/no-go decision versus the baseline; document as a model card; monitor drift and calibration in production.

Simplified pseudo-code:

```
load data -> apply cleaning log -> stratify_split(train, test)
for fold in StratifiedKFold(train):
   impute(median) + missingness_flags + onehot_encode   # fit on fold-train only
   fit logistic_regression(class_weight='balanced') and hist_gradient_boosting
   score fold-valid: PR-AUC, ROC-AUC, Brier, recall@threshold
select best model -> calibrate probabilities -> choose threshold by cost matrix
final evaluation on untouched test set -> fairness audit -> deploy with monitoring
```

Simplified implementation (scikit-learn):

```
pre = ColumnTransformer([('num', Pipeline([('imp', SimpleImputer(strategy='median')),
                                ('sc', StandardScaler())]), num_cols),
                ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)])
model = Pipeline([('pre', pre),
                ('clf', LogisticRegression(class_weight='balanced'))])
cross_validate(model, X, y, cv=RepeatedStratifiedKFold(5, 3),
           scoring={'pr_auc': 'average_precision', 'roc_auc': 'roc_auc'})
```

Baseline sanity check (executed on the cleaned Geldium data, 5x3 repeated stratified CV):

Model	***PR-AUC (mean ± sd)***	***ROC-AUC (mean ± sd)***	***Brier ↓***
Dummy (stratified)	*0.164 ± 0.011*	*0.492 ± 0.043*	*0.253*
Logistic Regression (balanced)	*0.169 ± 0.037*	*0.445 ± 0.052*	*0.259*
HistGradientBoosting	*0.150 ± 0.020*	*0.405 ± 0.048*	*0.167*
No-skill floor (delinquency base rate)	*0.160*	*0.500*	*-*

Table 1: Executed baseline check - all models sit at the no-skill floor, confirming the Task 1 finding that the delivered data carries no usable delinquency signal. The plan therefore includes an explicit go/no-go gate.

2. Justification for Model Choice

Explain why you selected this specific model type (e.g., logistic regression, decision tree, neural network). Consider:
- Accuracy
- Transparency
- Ease of use or implementation
- Relevance for financial prediction
- Suitability for Geldium’s business needs

Primary model: class-weighted, L2-regularised logistic regression. In consumer credit, the model must be defensible to risk managers, auditors and regulators as much as predictive. Logistic regression offers direct probability outputs, stable coefficients, and per-application reason codes (e.g., 'high debt-to-income ratio') that satisfy adverse-action notice requirements in financial services. With only 500 training records and ~16% positives, a linear, strongly regularised model is also the statistically appropriate choice - complex learners would fit noise. It is trivial to retrain, monitor and re-derive after data corrections, which suits Geldium's need to iterate quickly while its data quality issues are being resolved.

Considerations weighed:

- Accuracy: raw accuracy is rejected as a selection metric - at a 16% base rate a 'never delinquent' rule already scores 84%. Selection is by PR-AUC and recall at the operating threshold.

- Transparency: logistic regression yields explainable odds ratios and automatic reason codes; gradient boosting is retained only as a challenger and only if it beats the linear model materially and passes the same fairness and calibration gates (with SHAP-based explanations).

- Ease of use/implementation: a single scikit-learn pipeline (impute -> encode -> fit) reproduces training end-to-end, versioned with the cleaning log.

- Relevance for financial prediction: calibrated probabilities allow conversion into expected loss (probability x exposure via Loan_Balance), which is how Geldium would size interventions.

- Suitability for Geldium's business needs: risk-ranked queues with reason codes feed collections workflows directly; a challenger framework keeps room to gain accuracy later without changing the operating process.

Rejected alternatives: neural networks (far too little data, opaque, no regulatory justification); unweighted models (would under-predict the minority class); rule-only scorecards (premature while predictor-target relationships are unproven).

Key caveat: the Task 1 EDA found no statistically significant predictor-target association, and the executed baseline check (Table 1) confirms all models sit at the no-skill floor. Model selection is therefore provisional: if the go/no-go gate fails on refreshed data, the recommendation is to fix the data pipeline rather than to escalate model complexity.

3. Evaluation Strategy

Outline how you would evaluate your model's performance. Include:
- Which metrics you would use (e.g., accuracy, precision, recall, F1 score, AUC)
- How you would interpret those metrics
- Any plans to detect or reduce bias in your model
- Ethical considerations in making predictions about customer financial behavior

Evaluation is designed around a 16% positive base rate, financial consequences of errors, and the data-quality risks identified in Task 1. All metrics are computed on stratified cross-validation during development and once on the untouched hold-out test set for the final report.

Metrics and interpretation:

- Primary: PR-AUC (average precision) - informative under class imbalance; must exceed the no-skill floor of 0.16 by a material margin (gate: cross-validated PR-AUC clearly above floor with confidence intervals excluding it).

- Secondary: ROC-AUC for overall ranking quality; recall and precision at the cost-based threshold (a missed delinquent costs expected loss, a false alert costs an outreach call); F1 as a balance summary; lift/gains in the top risk decile to size campaign efficiency for collections.

- Calibration: Brier score and calibration plot - probabilities must be reliable before being multiplied by Loan_Balance exposure for expected-loss estimates.

- Confusion matrix and business-cost curve: choose the threshold that minimises total cost rather than maximising accuracy.

Bias detection and fairness:

- Subgroup analysis of recall, precision and calibration across Age bands (18-29/30-44/45-59/60+), Employment_Status and Location - the protected or proxy-sensitive attributes flagged in Task 1.

- Equal-opportunity gap (difference in true-positive rate across groups) and calibration parity; Location is additionally evaluated with-and-without to measure its marginal contribution versus its fairness risk.

- Small-cell guard: subgroup metrics reported only where n is sufficient; otherwise merged or suppressed to avoid noisy or disclosive statistics.

Ethical considerations:

- Predictions about financial behaviour can restrict credit access: models assist human collections officers, never trigger fully automated adverse decisions.

- Adverse-action transparency: every high-risk flag carries human-readable reason codes derived from the model; customers can be told why they were flagged.

- Data minimisation: protected attributes are used only for fairness auditing, not as scoring inputs, unless a documented business necessity and legal basis exist; inference of sensitive traits via proxies is explicitly tested.

- Governance: model card documenting training data, cleaning log, limitations (including the no-signal finding), drift monitoring (PSI) on inputs and outcomes, periodic recalibration, and a kill-switch back to manual review.

- Go/no-go gate: if the refreshed dataset still yields cross-validated PR-AUC at the no-skill floor, the model is NOT deployed; the finding is escalated to the data owner, because acting on noise would create unfair, arbitrary decisions.